

 迷宫小乌龟控制系统：修改内容与优化总结报告

一、 前端 index.html 的修改总结

前端主要围绕“自动模式的加入”和“手动/自动模式的控制权抢占冲突”进行了重构。

修改功能模块	修改前（原始状态）	修改后（当前终极状态）
UI 界面按钮	只有一个“重置到起点”的普通操作按钮。	在重置按钮旁新增了一个“开启/关闭自动寻路”按钮。
状态显示看板	仅显示 x, y, theta 等物理遥测数据。	遥测文本中新增了 mode（“自动寻路”/“手动控制”）的实时模式看板。
自动按钮交互	无此功能。	按钮状态与后端高度同步：处于自动寻路时，按钮变红并显示“关闭自动寻路”；处于手动控制时，按钮变绿并显示“开启自动寻路”。
手动按键拦截机制	没有任何限制，点击按键随时发送。	【终极优化】 当玩家在手机上按下 ↑ ↓ ← → 任意一个手动方向键时，前端会跳过等待， 无条件且立即 通过 WebSocket 向后端注入手动覆盖信号，强行切断并关闭自动状态，完全杜绝了手机按键失灵或乌龟原地锁死的情况。

二、 后端 turtlesim_web_bridge.py 的修改总结

后端是本次逻辑升级的核心，重点修复了**刚体碰撞死锁**、**坐标系卡死**以及**自动算法对接**的问题。

修改功能模块	修改前（原始状态）	修改后（当前终极状态）
算法模	没有任何自动驾	成功引入了 explorer.py（内部实现了 A* 路径

修改功能模块	修改前（原始状态）	修改后（当前终极状态）
块对接	驶、算法规划相关的代码。	规划算法），并实例化了全局 Planner 调度器。
自动寻路控制	无该状态，只接收网页端的手动单次指令。	在 compute_safe_motion 核心循环中接入了算法控制线：一旦 self.auto = True，便将当前的位姿状态输入给 A* 算法，由算法直接输出期望的线速度 linear 和角速度 angular。
重置逻辑优化	仅做 Turtlesim 物理位置的传送。	增加了算法重置逻辑：点击重置时，强制将 self.explorer.waypoints = None。 确保乌龟回到安全点后，算法能清空残余的错误路径，重新进行全局路径解算。
防卡死/转圈计时器（核心）	乌龟如果因为角度没对准路标，会在原地无限地疯狂转圈。	【新增防死锁算法】 增加了时间戳判定机制。如果在自动行驶时，同一个网格目标路标卡死移动超过 2.0 秒 （说明乌龟此时可能陷在黑墙缝隙里或者角度反复微调震荡）， 后端会强制将路标索引 idx += 1 递增，使其强行跳到下一格 ，完美实现了物理“脱困”和防原地打转。
手动接管与安全机制	撞墙后直接暴力将速度归零（safe_linear = 0.0），导致乌龟陷进墙里后彻底无法动弹。	1. 抢占式接管：只要收到 command 手动按键，self.auto 瞬间转为 False，人工控制永远具备最高优先级。 2. 碰撞容错处理：适当把安全判定半径 TURTLE_RADIUS 从 0.45 微调至 0.38，留出了一点物理缓冲区。并且移除了在手动控制下对线速度强制抹抹零的死锁，使得乌龟陷进墙后，玩家依然能够通过方向键“倒车”或“旋转”把乌龟安全开出来。

三、总结：修改后达到的效果

经过这几轮的修改和逻辑重构，整个项目从一个**纯手动长按的手机遥控器**，进化为了一个**具备高容错率、支持人机交互（手动/自动无缝切换）的智能走迷宫系统**。

1. 彻底解决了因为迷宫网格与 Turtlesim 物理坐标微小误差造成的“原地无限打转”死循环。
2. 彻底解决了因为安全锁死机制导致的“手机按键失灵”问题，只要人手一碰，乌龟立刻听话。